Transform Your Future with
"HELLO WORLD!"
INSTITUTE

Why learn with us ?

Training From Experts

One - on – One Guidance

100% Practical

Real-Time Projects & Case Studies

Placement Preparation Support

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Contact :- 8085896740 @ hello_world_institute Hello World Institute

Address : 2nd Floor , Near KFC , Tower Square , Bhawarkua , Indore

Functions

A function is a group of related statements that performs a specific task.

Functions help break our program into smaller and modular chunks. As our program grows larger and larger, functions make it more organized and manageable.

Furthermore, it avoids repetition and makes the code reusable.

Syntax of Function

```
Def  function_name(parameters):
"""docstring"""          statement(s)
```

Above shown is a function definition that consists of the following components.

1. Keyword **def** that marks the start of the function header.
2. A function name to uniquely identify the function. Function naming follows the same rules of writing identifiers in Python.
3. Parameters (arguments) through which we pass values to a function. They are optional.
4. A colon (:) to mark the end of the function header.
5. Optional documentation string (docstring) to describe what the function does.
6. One or more valid python statements that make up the function body.
   Statements must have the same indentation level (usually 4 spaces).
7. An optional **return** statement to return a value from the function.


OFFLINE CLASSES


ONLINE LECTURE


RECORDED ACCESS

Example of a function

```
def greet(name):                    #function definition
    """
    This function greets to     the
person passed in as     a
parameter
    """
    print("Hello, " + name + ". Good morning!")


greet('Paul')                       #function call
```

Types of Functions

Basically, we can divide functions into the following two types:

- **Built-in functions** - Functions that are built into Python.
- **User-defined functions** - Functions defined by the users themselves.

Function Arguments

You can call a function by using the following types of formal arguments –

- Required arguments
- Keyword arguments
- Default arguments
- Variable-length arguments

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Requires argument:

Required arguments are the arguments passed to a function in correct positional order. Here, the number of arguments in the function call should match exactly with the function definition.

```
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("hello", "world")
```

Keyword arguments:

Keyword arguments are related to the function calls. When you use keyword arguments in a function call, the caller identifies the arguments by the parameter name.

This allows you to skip arguments or place them out of order because the Python interpreter is able to use the keywords provided to match the values with parameters. You can also make keyword calls to the *printme()* function in the following ways –

```
# Function definition is here def
printinfo( name, age ):
   "This prints a passed info into this function"
print ("Name: ", name)    print ("Age ", age)
return;

# Now you can call printinfo function printinfo(
age=50, name="miki" )
```

Default argument:

A default argument is an argument that assumes a default value if a value is not provided in the function call for that argument. The following example gives an idea on default arguments, it prints default age if it is not passed –


OFFLINE CLASSES


ONLINE LECTURE


RECORDED ACCESS

```
# Function definition is here def
printinfo( name, age = 35 ):
   "This prints a passed info into this function"
print ("Name: ", name)    print ("Age ", age)
return;

# Now you can call printinfo function printinfo(
age=50, name="miki" ) printinfo( name="miki"
) printinfo("miki",34)
```

Variable-Length Arguments

We can use special characters in Python functions to pass as many arguments as we want in a function. There are two types of characters that we can use for this purpose:

1. ***args -**These are Non-Keyword Arguments
2. ****kwargs -** These are Keyword Arguments.

Arbitrary Arguments, *args

If you do not know how many arguments that will be passed into your function, add a * before the parameter name in the function definition.

This way the function will receive a *tuple* of arguments, and can access the items accordingly:

Example

If the number of arguments is unknown, add a * before the parameter name:

```
def my_function(*kids):

print("The youngest child is " + kids[2])

print("All Data “, kids)


my_function("Anvi", "Pihu", "Aashvi")
```

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Arbitrary Keyword Arguments, **kwargs

If you do not know how many keyword arguments that will be passed into your function, add two asterisk: ** before the parameter name in the function definition.

This way the function will receive a *dictionary* of arguments, and can access the items accordingly:

Example

If the number of keyword arguments is unknown, add a double ** before the parameter name:

```
def my_function(**kid):

 print("His last name is " + kid["lname"])
print("All Data “, kid)


my_function(fname = "Love", lname = "kush")
```

Passing a List as an Argument

You can send any data types of argument to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.

E.g. if you send a List as an argument, it will still be a List when it reaches the function:

Example

```
def my_function(fruit):

  for x in fruit:
    print(x)

fruits = ["apple", "banana", "cherry"]

my_function(fruits)
```

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Pass by Reference or pass by value

One important thing to note is, in Python every variable name is a reference.
When we pass a variable to a function, a new reference to the object is created.
Parameter passing in Python is the same as reference passing in Java.

```
# Here x is a new reference to same list lst

def myFun(x):

        x[0] = 20

lst = [10, 11, 12, 13, 14, 15]

myFun(lst)

print(lst)
```

The *Anonymous* Functions

These functions are called anonymous because they are not declared in the standard manner by using the *def* keyword. You can use the *lambda* keyword to create small anonymous functions.

- Lambda forms can take any number of arguments but return just one value in the form of an expression. They cannot contain commands or multiple expressions.
- An anonymous function cannot be a direct call to print because lambda requires an expression
- Lambda functions have their own local namespace and cannot access variables other than those in their parameter list and those in the global namespace.
- Although it appears that lambda's are a one-line version of a function, they are not equivalent to inline statements in C or C++, whose purpose is by passing function stack allocation during invocation for performance reasons.

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Syntax

The syntax of *lambda* functions contains only a single statement, which is as follows –

```
lambda [arg1 [,arg2,.....argn]]:expression
```

Following is the example to show how *lambda* form of function works –

```
# Function definition is here
 sum = lambda arg1, arg2: arg1 + arg2;

# Now you can call sum as a function
Print("Value of total : ", sum( 10, 20 ))
Print("Value of total : ", sum( 20, 20 ))
```


OFFLINE CLASSES


ONLINE LECTURE


RECORDED ACCESS